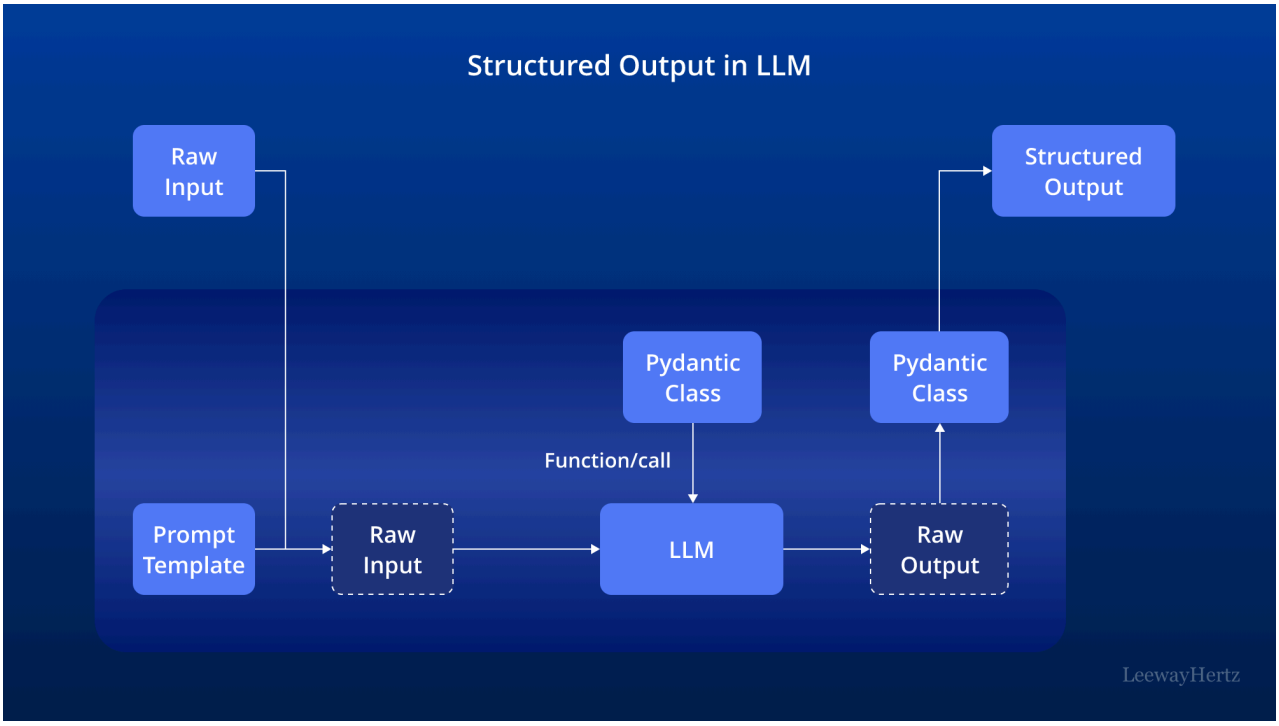
Structured outputs in LLMs: Definition, techniques, applications, benefits

Talk to our consultant



Listen to the article

00:00 31:43

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

Large Language Models (LLMs) have transformed our engagement with artificial intelligence, enabling more natural and intuitive communication by generating human-like text and grasping complex linguistic nuances. As per Polaris Market Research, the global market for large language models (LLMs) is projected to reach USD 61.74 billion by 2032. These models, built on massive datasets and advanced neural network architectures, can produce coherent, contextually appropriate, and often impressively creative responses across diverse topics and queries. LLMs have showcased their versatility in handling various tasks requiring a deep understanding of language, from drafting emails to creating detailed reports or engaging in conversational dialogue.

However, while LLMs excel in generating free-form, natural language text, this strength can also present challenges when there is a need for specific, consistent, and structured information. In many real-world applications, particularly in sectors like finance, healthcare, and legal services, producing outputs that adhere to strict formatting rules is crucial. This is where the concept of structured outputs from LLMs becomes critically important. Structured outputs refer to the capability of these models to generate content that conforms to predefined formats, such as JSON, XML, or other structured data schemas. This approach ensures that the information produced is relevant, accurate and organized, making it easy to interpret and integrate into existing systems, such as databases, APIs, and analytical tools.

By combining the linguistic capabilities of LLMs with the precision and predictability of structured data, organizations can achieve new levels of efficiency and reliability in their AI-driven processes. This synergy allows the automation of complex tasks that require both deep language understanding and strict adherence to format, enabling more seamless integration of AI into business operations. Whether it's generating financial reports, processing legal documents, or analyzing healthcare data, the

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

- Traditional LLM outputs vs. structured outputs
- How do structured outputs from LLMs are generated?
- Understanding structured output functions
- Applications of structured outputs in LLMs
- Techniques for generating structured outputs
- How to implement JSON schema for the structured response?
- How do constrained sampling and context-free grammar (CFG) ensure accurate structured outputs?
- Benefits of structured outputs from LLMs

Definition and capabilities of structured outputs in LLMs

Structured outputs in large language models (LLMs) refer to the ability of these models to generate responses in specific, predefined formats rather than just free-form text. This capability allows LLMs to produce more organized, consistent, and machine-readable outputs that can be easily parsed and used in various applications. A few examples include:

- The chatbot should utilize quotation marks for dialogue, employ special symbols for scene descriptions, and adhere to a specific format for listing movie characteristics. This approach ensures that the output is easily parsed and used for training.
- Classify student answers as right, wrong, or uncertain. For sentiment analysis, restrict output to fixed classes such as positive, negative, neutral, and strongly positive.
- Summarize each point to under 40 words. If a summary exceeds this limit,

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

means they produce natural language output that, while coherent and informative, doesn't follow a strict structure or format. For example, if you ask an LLM about the weather, it might respond with a paragraph describing current conditions, forecasts, and general weather-related information.

2. **Structured outputs**: In contrast, structured outputs are responses generated by LLMs that adhere to a specific, predefined format or structure. This structure can take various forms, such as:

- JSON or XML data
- Filled templates or forms
- Tabular data
- Specific answer formats (e.g., multiple-choice, true/false)
- Parsed entities or relationships

3. **Predefined formats:** The "predefined" aspect is crucial. It means that before the LLM generates a response, there's an agreement or specification on what the output should look like. This could be as simple as always providing a yes/no answer, followed by an explanation, or as complex as generating a fully structured JSON object with specific keys and value types.

The key capabilities of structured outputs in LLMs include:

Machine-readability: One key advantage of structured outputs is that they're easily readable by machines. While humans can interpret free-form text, computers work much better with structured data. This makes integrating LLM outputs directly into other software systems or databases easier.

Parsing and application: Because structured outputs follow a known format, the software can easily parse them (break them down into their constituent parts). This parsing allows for quick extracting of relevant information and facilitates using LLM

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

required by many computer systems and applications. They allow us to harness the power of LLMs in more controlled and application-specific ways, opening up new possibilities for integrating AI-generated content into various workflows and systems.

Traditional LLM outputs vs. structured outputs

Here's a comparison of traditional LLM outputs versus structured outputs :

Aspect	Traditional LLM outputs	Structured outputs
Output format	Free-form text, unstructured, and often variable in structure.	Conforms to predefined formats like JSON, XML, or other structured schemas.
Reliability	May vary in structure; prone to inconsistencies and errors.	Consistently follows the specified schema, ensuring predictability.
Error handling	Errors and omissions are common and require manual validation or retries.	Minimizes errors with reliable type-safety and structured data formats.
Prompt complexity	Requires detailed and strongly worded prompts to achieve specific formats.	Simplified prompting, as the structure is enforced by the schema.
Data validation	Often requires post-	Built-in validation through

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

downstream applications	integrate due to variable output formats.	downstream applications that require structured data.
Handling of missing data	May omit required information, leading to incomplete outputs.	Ensures all required fields are present, as defined by the schema.
Handling of invalid values	Prone to generating incorrect or unexpected values (e.g., invalid enums).	Ensures that only valid values, as defined by the schema, are produced.
Detecting safety issues	Safety issues or refusals may be ambiguous and hard to detect.	Explicit and programmatically detectable refusals or safety issues.
Use cases	Suitable for creative or open-ended tasks where flexibility is needed.	Ideal for tasks requiring structured, predictable, and machine-readable outputs.
Implementation complexity	Simpler to implement but requires more effort in managing output quality.	More complex to set up but results in higher quality and consistent outputs.
Integration with APIs	May require custom parsers or extra steps to format data for API use.	Directly usable with APIs that accept structured formats like JSON or XML.

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

LLMs generate text one token at a time. A token can be a word, part of a word, or even a punctuation mark. At each step, the model considers all possible tokens and assigns a probability to each one based on how likely it is to be the correct next token in the sequence. The model selects a token according to this probability distribution and generates the next token.

Structured output generation

When we need the LLM to generate text in a specific format, like JSON, we can't just let it pick any token. We need to ensure that every token it selects follows the rules of that format. This is where the concept of a Finite State Machine (FSM) comes into play.

What is a Finite State Machine (FSM)?

The Finite State Machine (FSM) plays a crucial role in guiding the generation process of a Large Language Model (LLM) when producing structured outputs. Here's how it works:

As the LLM generates text, it does so one token at a time, progressing through various states in the FSM. Each state represents a specific point in the generation process, corresponding to a part of the desired structure (e.g., JSON or XML format). The FSM determines which transitions, or token choices, are valid for each state based on the predefined structure. By calculating these permissible transitions, the FSM can identify which tokens are acceptable as the "next step" in the sequence.

During the decoding phase, the FSM tracks the current state of the LLM's output. It then filters out any tokens that don't fit the required format by applying a technique known as logit bias. This involves adjusting each token's probabilities (logits),

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

1. **Defining the format:** We start by specifying the format we want, using a JSON schema that defines the structure and rules for the output.
2. **Building the FSM:** The FSM is built as a directed graph, where each node represents a valid partial sequence of tokens. The edges connecting the nodes represent the valid next tokens that can follow.

Ensuring structured output

When the LLM is generating text, instead of selecting tokens based on the usual probability distribution, we modify this step:

Filtering the tokens: The FSM checks which tokens are valid based on the current state. The model then adjusts the probabilities, setting the likelihood of invalid tokens to zero. This ensures that the model only chooses valid tokens that follow the format.

This process must be fast and efficient, especially when generating long or complex outputs. Some open-source tools can help create structured outputs but often slow down the model's performance. To avoid this, engineering optimizations were implemented to make the FSM construction and token filtering much faster. These optimizations ensure the model can enforce the structure without slowing down its text generation.

Using an FSM and optimized sampling strategies, we can reliably and efficiently guide LLMs to produce structured outputs. This process ensures that the generated text adheres to the desired format, making it suitable for integration with other systems or applications where consistency and structure are crucial.

Understanding structured output functions

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

2. **Prompt template:** A prompt template is applied to the raw input. This template guides the LLM in processing the input and producing the expected output. The prompt might include instructions for returning the output in a particular format.
3. **Generic LLM with formatting instructions:** The prompt, combined with the raw input, is passed to a generic LLM. The LLM processes the input according to the prompt's instructions. However, since this approach uses a generic completion API, the model may not always perfectly adhere to the structured format.
4. **Raw output:** The LLM generates a raw output based on the input and prompt. This output is not guaranteed to be perfectly structured.
5. **Pydantic output parser (Initial processing):** The raw output is then processed using a Pydantic OutputParser. Pydantic is a data validation and parsing library in Python that helps enforce a schema on the output, ensuring it adheres to the expected format.
6. **Structured output:** After processing with Pydantic, the output is transformed into a structured format. This ensures the output structure remains consistent and dependable.

Function calling API workflow:

We will understand this with the help of Open AI's example:

1. **Raw input:** Similar to the generic workflow, it starts with raw input data that needs to be structured.
2. **Prompt template:** A prompt template is applied to this input, guiding the LLM on what is expected. The difference here is that the LLM is equipped with the ability to perform function calls.
3. **Function call:** Instead of merely generating text, the LLM makes a function call based on the input and the prompt template. This function is predefined to handle

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

6. **Structured output:** Finally, the processed output is presented in a structured format, ensuring it meets the task's specific requirements.

Comparison between the two approaches:

- **Generic completion API:**
  - This approach is more flexible and works with any generic LLM. However, validating and enforcing the output's structure may require additional steps.
- **Function calling API:**
  - This method is more robust, using predefined functions to ensure the output adheres to a specific format. It is more reliable for producing structured outputs directly and reduces the need for extensive post-processing.

The key difference lies in how strictly the output format is enforced. The function-calling API provides more precise control over the structure, while the generic completion API offers more flexibility at the cost of potential inconsistency.

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

Applications of structured outputs in LLMs

Let's explore some common applications of structured outputs in LLMs

1. **Data extraction and organization**: Structured outputs enable LLMs to parse and organize unstructured text into predefined formats such as tables or spreadsheets. This application is particularly useful for extracting key data from documents, emails, or web pages, making it easier to analyze and integrate information into databases or reporting systems. For example, LLMs can transform research papers into

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

organized formats highlighting essential information by providing structured summaries. This approach can include bullet points, categorized summaries, or executive summaries, helping users quickly grasp the key points without sifting through extensive text. For example, LLMs can summarize research papers into structured abstracts with sections for methodology, results, and conclusions.

4. **Question answering systems**: Structured outputs in LLMs allow for precise and organized responses to user queries. Instead of generating free-form text, the model can deliver answers in lists, tables, or categorized formats, enhancing clarity and ease of understanding. For instance, an LLM could answer a customer support question by providing a step-by-step troubleshooting guide in a structured format.

5. **Knowledge base management**: LLMs can help build and maintain structured knowledge bases or FAQs by extracting relevant information and organizing it into predefined categories. This structured approach ensures that information is accessible and searchable, improving the efficiency of knowledge management systems. For example, an LLM can update a company's knowledge base with new procedures or policies in a well-organized format.

6. **Report generation**: LLMs can automatically generate reports with consistent formats and sections using structured outputs. This is useful for creating business intelligence reports, performance summaries, or financial statements. For example, an LLM can produce a monthly sales report with sections for sales metrics, trend analysis, and executive summary, adhering to a predefined template.

7. **Compliance and regulation**: Structured outputs help ensure that regulatory and compliance information is presented clearly and standardized. LLMs can compile legal documents, compliance checklists, or policy guidelines into structured formats, making reviewing and adhering to regulations easier. For example, an LLM can create a compliance report with sections for regulatory requirements, action items, and evidence of compliance.

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

guides. This structured approach helps agents and customers quickly find solutions and ensures consistency in support interactions. For example, an LLM could generate a structured FAQ with categorized questions and answers based on customer inquiries.

10. **Content classification**: Structured outputs assist in categorizing and tagging content for better management and retrieval. LLMs can classify articles, documents, or media into predefined categories, making organizing and searching for content easier. For example, an LLM could classify news articles into topics like politics, sports, and technology, enhancing content organization and retrieval.
11. **Code generation:** Structured outputs allow LLMs to create precise code snippets or complete programs, ensuring they adhere to specific structures and formatting guidelines, which is crucial for maintaining code quality and functionality.
12. **Language translation:** By utilizing structured outputs, LLMs can maintain the original formatting and structure when translating text between languages, ensuring that the translated content remains clear and consistent with the source material.
13. **Automated reasoning:** Structured outputs enable LLMs to perform tasks that require logical reasoning or problem-solving, allowing them to break down complex problems into manageable steps and produce well-organized solutions.
14. **Data augmentation:** LLMs can generate structured data that serves as synthetic training data for other machine learning models or test data for validating algorithms, enhancing the robustness and diversity of data sets.

Techniques for generating structured outputs

Generating structured outputs with LLMs can be achieved through various techniques, each offering different levels of precision and flexibility. These methods help produce data in specific formats, ensuring that outputs are well-organized and consistent.

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

Prompting

Prompting involves carefully creating a prompt that requests the LLM generate output in a specific format, such as JSON or XML. This method is universally applicable across different LLMs and doesn't require special configurations. However, its effectiveness relies heavily on how well the prompt is designed, and there's no assurance that the model will consistently adhere to the desired format. As a result, while prompting is a flexible and straightforward approach, it may occasionally yield

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

approach involves showing a few sample inputs and their corresponding outputs, helping the model understand and replicate the correct formatting in its responses.

However, its effectiveness relies heavily on how well the prompt is designed, and there's no absolute assurance that the model will consistently adhere to the desired format. As a result, while prompting is a flexible and straightforward approach, it may occasionally yield inconsistent or unstructured responses.

Function calling

Function calling allows an LLM to generate specific function calls based on the prompt and additional parameters provided. This technique typically involves passing function names and descriptions to the model, enabling it to determine the appropriate actions during the task completion. Function calling enhances the model's ability to perform targeted tasks and return structured output. However, it requires fine-tuning or specific model support, and the inclusion of function-related details can increase token usage, impacting the overall cost and efficiency of the operation.

Tool calling

Tool calling extends the concept of function calling by enabling the LLM to invoke multiple functions or tools simultaneously. This approach empowers the model to handle more complex tasks by leveraging a combination of tools as needed. Tool calling enhances versatility and allows dynamic interactions within the model's environment. Like function calling, this technique requires specialized support and can lead to higher token counts, but it offers greater flexibility and the ability to execute more intricate operations.

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

cases or reduce the model's flexibility in other types of output generation.

Therefore, each technique offers unique advantages for generating structured outputs with varying degrees of precision and flexibility. The next section will discuss the practical steps to implement JSON schema for effectively structuring outputs.

How to implement JSON schema for the structured response?

To implement a JSON schema for extracting structured data from text using a Large Language Model (LLM), such as GPT-4, you can follow a series of steps to ensure that the output adheres to a predefined format. Here, the task is identifying and categorizing named entities (like people, organizations, and dates) from a text. Here's how the process works:

Define the JSON schema

First, create a JSON schema that outlines the data structure you want the LLM to return. A JSON schema is a blueprint that defines the output's data types, structure, and constraints.

In this example, the JSON schema defines an object called named entities, which contains an array of entities. Each entity is represented as an object with two properties:

- **Name:** The actual name in the text, such as "Samuel Harris Altman" or "OpenAI."
- **Type**: The category of the entity, such as "Person," "Organization," "Location," or "DateTime."

The schema also specifies that the name must be a string, and the type must be one of the predefined categories (Person, organization, location, datetime). This ensures

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

- **Required fields**: The schema specifies that each entity's name and type are required. This means the model must provide both fields for every entity it identifies.
- **Optional fields**: In this approach, you can create "optional" fields by defining them to accept either a string or a null value. This allows these fields to be left empty if needed, providing flexibility in the data structure.

Provide instructions to the model

Once the schema is defined, the next step is to instruct the LLM on how to use it. The instructions include details about the specific task (in this case, identifying named entities) and any rules the model should follow. For instance, the model is told to ignore certain acronyms like "CEO" or "CTO" and only to return entities that fit one of the predefined categories.

Integrate the schema with the LLM

After defining the schema and instructions, you must integrate them with the LLM. This is done by passing the schema, text data, and instructions into the model's API. The schema guides the model in generating the output in the correct format, ensuring that the results are structured according to the blueprint you provided.

Generate and interpret the output

The LLM processes the input text and generates a response that conforms to the JSON schema. The output is a structured list of entities with names and types. For example:

- "Samuel Harris Altman" is classified as a "Person."
- "April 22, 1985" is identified as a "DateTime."
- "OpenAI" is categorized as an "Organization."

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

other systems. Additionally, it allows you to enforce strict requirements, ensuring that all necessary information is included in the output.

How do constrained sampling and context-free grammar (CFG) ensure accurate structured outputs?

Generating structured outputs in a large language model (LLM) relies on constrained sampling and a formal structure known as Context-Free Grammar (CFG). This technique ensures that the model adheres strictly to predefined formats, such as JSON schemas, by selectively guiding the generation of each token.

Constrained sampling

In typical LLM operations, the model generates text one token at a time, choosing from a large vocabulary (often over 40,000 tokens) based on learned probabilities. Since most of the pre-training data is in natural language, there is always a chance that the model might select an irrelevant token during any given step. However, minimizing the likelihood of selecting inappropriate tokens is crucial when working with structured formats like a JSON schema.

For instance, if the model has already generated part of a structured output, such as:

{'entities': [{'name': 'Samuel Harris Altman',

the next token should logically complete the key-value pair, such as 'type.' At this point, you want the model to favor selecting the appropriate token ('type') and reduce the probability of all other tokens. This is where constrained sampling comes into play—adjusting the probability (or logit bias) to prefer tokens that strongly fit the required structure.

Context-Free Grammar (CFG)

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

conform to the expected structure are considered viable options.

Let's clarify this with an example: we're using CFG (Context-Free Grammar) to generate a simple sentence structure. Our grammar rules might define that a sentence must consist of a subject, followed by a verb, and then an object.

In this case, when generating a sentence, CFG would first look for a word that fits the "subject" category – let's say it could be "The cat" or "A dog." According to our rules, once a subject is chosen, the next token must be a verb. So words like "jumps", "runs", or "sleeps" would be viable options, while nouns like "ball" or adjectives like "furry" would be excluded at this point.

After the verb, our grammar requires an object. So phrases like "the fence" or "a mouse" would be considered, while any other verbs or subjects would be ruled out.

This process ensures that we end up with grammatically correct sentences like "The cat chases a mouse" or "A dog jumps the fence" rather than irrelevant combinations like "Cat the chases mouse a" or "Jumps dog fence the."

While CFG has been a foundational concept in computer science and natural language processing (NLP) for years, it's now being applied in modern LLMs to enhance the accuracy and reliability of structured outputs. Using CFG, the model can effectively guide its token generation process, drastically reducing the chance of producing incorrect or out-of-place tokens. This approach is especially useful when dealing with formal schemas where precise output is critical.

The combination of constrained sampling and CFG ensures that LLMs can generate structured outputs that strictly adhere to predefined formats, significantly improving the reliability and accuracy of the results. This process eliminates the risk of selecting inappropriate tokens, ensuring that each step in the text generation process aligns

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

responses are accurate and consistent by confining the model to a specific format. This minimizes the risk of including irrelevant or inconsistent information, leading to more reliable results.

Enhanced interpretability: Structured data is easier to interpret for both humans and machines. It allows quick extraction of key information without sifting through lengthy or convoluted text, making the data more actionable and clear.

Easier integration with systems: Structured responses streamline the integration process with other software systems. Databases, APIs, or applications can directly ingest structured data, simplifying data handling and reducing the need for complex natural language processing.

Reduced hallucinations and irrelevant information: Specifying a structure helps the LLM avoid generating extraneous or incorrect information. Adhering to a defined format makes the model less likely to "hallucinate" facts or include irrelevant details.

Streamlined data processing and analysis: Structured responses facilitate efficient data processing, analysis, and visualization. For example, collecting customer preferences in a structured format allows for rapid aggregation and insight generation, enhancing overall data utility and decision-making.

Endnote

Structured outputs from Large Language Models (LLMs) ensure that AI-generated content meets specific formatting and accuracy requirements. By employing methods such as prompt engineering, function calling, and JSON schema enforcement, these models can produce consistent, reliable and easily interpretable outputs by humans and machines. This structured approach reduces the risk of errors and irrelevant information, known as "hallucinations," and prepares the data

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

needed to implement AI-driven solutions. This advancement accelerates innovation and ensures that AI technologies can be effectively and reliably integrated into everyday business operations, making them indispensable tools for informed decision-making and improved productivity.

Unlock the power of structured outputs in LLMs—discover how they can transform your data management and analysis today! Contact Leewayhertz experts for your requirements.

Listen to the article

00:00 31:43

Author's Bio

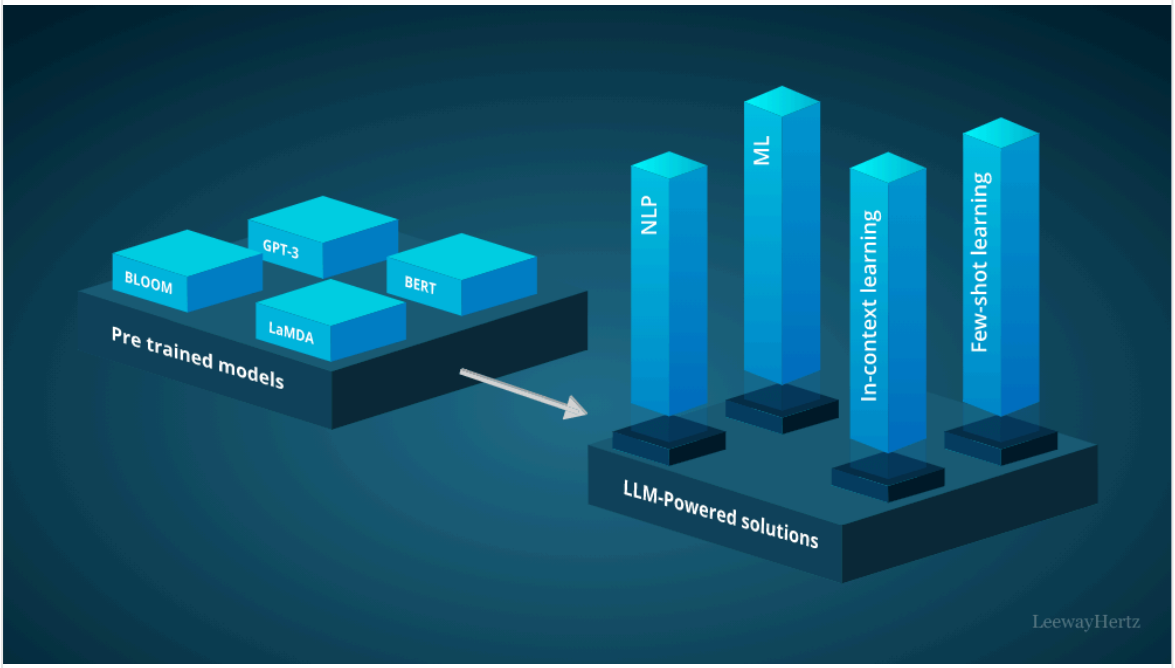
Akash Takyar
CEO LeewayHertz

Akash Takyar is the founder and CEO of LeewayHertz. With a proven track record of conceptualizing and architecting 100+ user-centric and scalable

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

Related Services



LLM Development

Transform your AI capabilities with our custom LLM development services, tailored to your industry's unique needs.

Explore Service

Start a conversation by filling the form

Once you let us know your requirement, our technical expert will schedule a call and discuss your idea in detail post sign of an NDA.

All information will be kept confidential.

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

Select a country*

Comments*

0 of 300 max characters

I agree to the Terms of use and Privacy policy.

Start a conversation

Insights

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

AI in debt collection: Scaling efficiency and customer experience

AI is a transformative solution for the debt collection process, enhancing the entire ecosystem by driving efficiencies, reducing risks, and fostering better relationships between creditors and debtors.

AI in Project Management: Use cases, benefits, techniques, solution and implementation

AI can enhance project management by automating repetitive tasks, providing data-driven insights, improving decision-making and more.

Machine learning for customer segmentation: Unlocking new dimensions in marketing

Customer segmentation is a powerful technique that helps businesses understand their customer base better and tailor their marketing strategies accordingly.

Show all insights

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

Related Functional Agents

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

Legal AI Agents

ZBrain AI Agents for Legal Operations streamline complex workflows by automating contract management, compliance tracking, risk assessment, and document organization, enhancing accuracy and efficiency while allowing legal teams to focus on strategic decisions.

Operations

Operations AI Agents

ZBrain AI Agents for Operations automate order management, task creation, SLA analysis, spend analytics, and technical interpretation, boosting efficiency, reducing manual work, and enabling teams to focus on strategic priorities.

Marketing

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings
Accept All Cookies

About Us
Careers
Case Studies
Work
Community
Privacy Policy
Rackspace
URC
Scrut Automation
NSG
AdPerfect
ZBrain
AI Development
AI Consulting
Software Development
Hire AI Developers
AI Agents for Finance
AI Agents for Sales
AI Agents for Customer Service
AI Agents for IT
AI Agents for Legal
AI Agents for Marketing

PRODUCTS

ZBrain AI XPLR™
ZBrain Builder
ZBrain Agents
Agent Store

INSIGHTS

Vibe Coding
Agentic AI
Context Engineering in ZBrain
Architecting Resilient AI Agents
A2A Protocol
ZBrain Agent Crew

CONTACT US

Get In Touch
sales@leewayhertz.com
jobs@leewayhertz.com

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings Accept All Cookies

The Hackett Group® is a leading digital transformation and AI strategy consulting firm with expertise in advanced analytics, business benchmarking, and intelligent automation. Leveraging our renowned benchmarking and best practices database and our revolutionary Hackett AI XPLR™ tool, we provide fact-based advice that drives AI for business, as well as continuous improvement in cost optimization, service quality and human productivity across the enterprise. We can design your IT strategy for cloud services, data and analytics, human capital management, and eProcurement, including Oracle implementation, SAP implementation, Coupa implementation and more.

Copyright © 2026 The Hackett Group, Inc. All Rights Reserved.

This website uses cookies to personalize content, analyze our traffic and enhance your experience. For information on what cookies, we use visit our cookie policy. For information on how we utilize personal information that we collect, please see our privacy statement.

Cookie settings
Accept All Cookies